Durham University

Version Control: *Introduction*

Chris Marcotte[1]
Durham University

[1] For errata and questions: christopher.marcotte@durham.ac.uk

Overview

Outline

Motivation 3
Version Control History 6
Versioning Model 10
Outlook 16

Overview

Outline

Motivation 3
Version Control History 6
Versioning Model 10
Outlook 16

Learning Goals

- Recognize the utility of version control
- Understand the basics of the versioning model
- Be able to employ version control skills — especially for your project!

Motivation

Version Control

In practice, version control is:

- a better way to manage changes to a codebase,
- a better way to collaborate than sending files, and
- a better way to share your code and other work with the world.

Version Control is a method of tracking changes to a directory containing subdirectories and files, over time, across potentially many contributors.

Why use Version Control?

Who has wound up with a file called

`1A_1a_final_FINAL_version_rewrite_fINAL.docx.pdf`?

Using a software tool to handle versioning of your project files frees you to work on the stuff that *matters* – solving problems.

Why use Version Control?

Who has wound up with a file called

`1A_1a_final_FINAL_version_rewrite_fINAL.docx.pdf`?

Using a software tool to handle versioning of your project files frees you to work on the stuff that *matters* – solving problems.

Versioning is a habit with many ancillary benefits:

- A repository is a full backup of a project and its history
- Simplified integration of changes from multiple contributors
- GitHub hosting provides a web-based collaboration service
- Version control is a *de facto* standard in software development

Version Control History

Centralized Versioning Systems

Early version control systems were centralized, meaning that all changes were tracked on a single server.

Examples include:

- **RCS (Revision Control System)** *not* the replacement for SMS
- **CVS (Concurrent Versions System)** *very* annoying to use
- **Subversion (SVN)** merely annoying to use

These systems were often difficult to use and manage, and they were not well-suited for distributed development.

The centralized structure ensured that all changes are coherent, but introduce a bottleneck for collaboration and scalability.

Distributed Versioning Systems

Distributed version control systems allow each developer to have a complete copy of the repository on their local machine.

Examples include:

- **Git** the focus of the rest of today's workshop
- **Mercurial** the runner-up in terms of popularity and features
- **GNU Bazaar / Breezy** literally never heard of it

Distributed version control systems are more flexible and scalable than centralized systems, and they are better suited for distributed development.

We will focus on Git, which is the most popular distributed version control system; however Mercurial is not bad and you may use it in your career.

Version Control Differences

Feature	Subversion	Mercurial	*Git*
File Storage	File deltas against previous version	File deltas against previous version	Complete file snapshot, compressed and deduplicated
Locality	Central server holding *truth* repository	Local offline repositories supported, decentralized model.	Local-first, distributed model.
Integrity	Checksum-based integrity checks; centralized repository creates single failure point.	SHA-1 for identity resolution, checksum for object integrity	SHA-1 for identity resolution, checksum for object integrity
Recovery	Commits replace prior state in the central repository; destructive updates	Phases (draft, public, secret) to prevent accidental publication; not append-only.	By default, append-only, hard to lose data; reflog is comprehensive
File States	1. Unversioned 2. Modified 3. Normal	1. Untracked 2. Modified 3. Staged 4. Committed	1. Untracked 2. Modified 3. Staged 4. Committed
Speed	Slow, scales poorly	Slow, scales well	Fast, scales well

Versioning Model

Directed Acyclic Graphs (DAGs)

Version Control can be understood as a Directed Acyclic Graph (DAG):

- whose *direction* is inherited from the flow of time (past ← future)
- with *no cycles*, so future revisions are always distinct
- and potentially multiple trunks (product versions)

Directed Acyclic Graphs (DAGs)

Version Control can be understood as a Directed Acyclic Graph (DAG):

- whose *direction* is inherited from the flow of time (past ← future)
- with *no cycles*, so future revisions are always distinct
- and potentially multiple trunks (product versions)

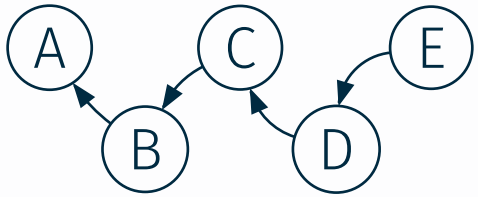
Generally, Version Control describes a main *trunk* with **branches** and **merges**.

Branches are where an iteration points to more than one future iteration.
Merges are where an iteration is pointed to by more than one past iteration.

You are not *required* to have branches and merges to use version control – but these provide a structured way to manage changes and collaborate with others.

Version Control DAG

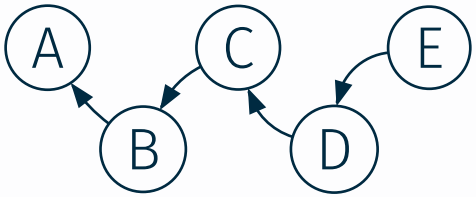
In the simplest case, a version control DAG proceeds directly from a past commit to a future commit without any branching or merges.[1]

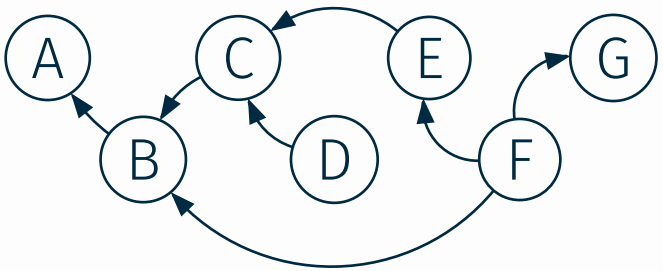


[1] Traditional *Git* DAGs are drawn latest commit at the top, and arrows point *from* future commits *to* past ones.

Version Control DAG

In the simplest case, a version control DAG proceeds directly from a past commit to a future commit without any branching or merges.[1]

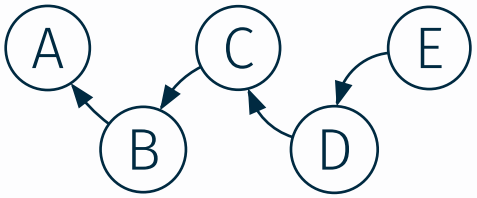




1. Initial commit “A” updates to “B”,
2. Commit “B” produces “C”,
3. Commit “C” produces “D”,
4. Commit “D” is abandoned,
5. The end-deliverable (“F”) has
   - early contributions (“B”),
   - late contributions (“E”), and
   - external contributions (“G”) (e.g. a *pull request*)

[1] Traditional *Git* DAGs are drawn latest commit at the top, and arrows point *from* future commits *to* past ones.

Cyclic Graphs are not Version Control Histories

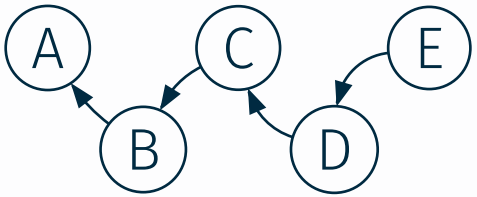


This is the basic structure of a sequential *Git* graph.

E → D, which → C, which → B, which → A.

There are no branches, no merges, *impossible* to have cycles.

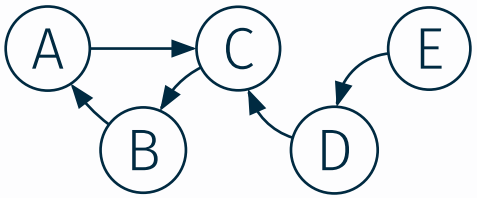
Cyclic Graphs are not Version Control Histories



This is the basic structure of a sequential *Git* graph.

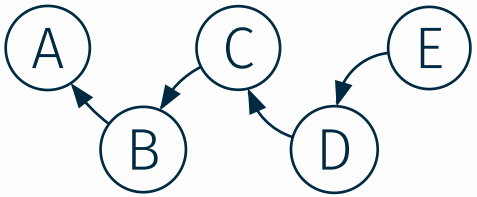
E → D, which → C, which → B, which → A.

There are no branches, no merges, *impossible* to have cycles.



Adding a single connection so that A → C creates a A-B-C cycle so that C occurs both *before* **and** *after* A.

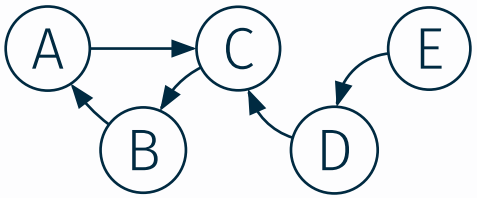
Cyclic Graphs are not Version Control Histories



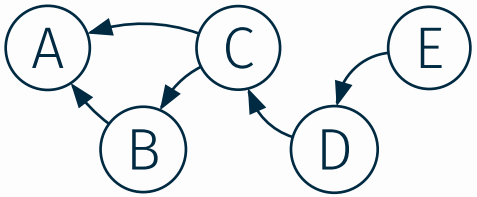
This is the basic structure of a sequential *Git* graph.

E → D, which → C, which → B, which → A.

There are no branches, no merges, *impossible* to have cycles.



Adding a single connection so that A → C creates a A-B-C cycle so that C occurs both *before* **and** *after* A.



Adding a single connection so that C → A does not create a cycle, this *can* be a version control history.

Example: Version Control Graphs

There are a number of branching strategies in *Git* – details in a later lecture.



Git-graph'd history of the git-graph repository.[1]

[1] Note: a more utilitarian and less graph-like output can be browsed with `git log --oneline --decorate --graph --all`.

Example: Version Control Graphs

There are a number of branching strategies in *Git* – details in a later lecture.



Git-graph'd history of the git-graph repository.[1]



Git-graph'd history of the Parallel Data Assimilation Framework repository.

[1] Note: a more utilitarian and less graph-like output can be browsed with `git log --oneline --decorate --graph --all`.

Consistency Model

Consistency Model is a formal specification that through permitted operations on an object, it can be made consistent; e.g. like a shared Word document.

[1] If you search for “git consistency model” you will get diffusion-type models for image generation hosted on GitHub.

[2] You have used CRDTs in any software which tries to provide near-real-time multi-editing capabilities.

Consistency Model

Consistency Model is a formal specification that through permitted operations on an object, it can be made consistent; e.g. like a shared Word document.

Git does not actually provide a coherent consistency model[1] – this is frequently a pain point for new users, and something we'll discuss.

---

[1] If you search for "git consistency model" you will get diffusion-type models for image generation hosted on GitHub.

[2] You have used CRDTs in any software which tries to provide near-real-time multi-editing capabilities.

Consistency Model

Consistency Model is a formal specification that through permitted operations on an object, it can be made consistent; e.g. like a shared Word document.

Git does not actually provide a coherent consistency model[1] – this is frequently a pain point for new users, and something we'll discuss.

There is research consistency for distributed systems, which may inform future version control software, using Conflict-free Replicated Data Types (CRDT).

CRDT is a datatype which provides strong guarantees of *eventual* consistency for an underlying state or operation sequence[2].

---

[1] If you search for "git consistency model" you will get diffusion-type models for image generation hosted on GitHub.

[2] You have used CRDTs in any software which tries to provide near-real-time multi-editing capabilities.

Outlook

Rest of this Session

The rest of today is broken into three parts:

Command Line *Git*

- making a new repository
- branching and merging
- conflict resolution
- typical workflows

Rest of this Session

The rest of today is broken into three parts:

Command Line *Git*

- making a new repository
- branching and merging
- conflict resolution
- typical workflows

GitHub & Collaboration

- making a GitHub account
- connecting a repository
- responsible file management
- distributed workflows

Rest of this Session

The rest of today is broken into three parts:

Command Line *Git*

- making a new repository
- branching and merging
- conflict resolution
- typical workflows

GitHub & Collaboration

- making a GitHub account
- connecting a repository
- responsible file management
- distributed workflows

Summary & Task

- summarizing version control
- key skills
- **Workshop Task**

Rest of this Session

The rest of today is broken into three parts:

Command Line *Git*

- making a new repository
- branching and merging
- conflict resolution
- typical workflows

GitHub & Collaboration

- making a GitHub account
- connecting a repository
- responsible file management
- distributed workflows

Summary & Task

- summarizing version control
- key skills
- **Workshop Task**

We expect you to follow along so you can use these skills in the programme.

Additional Resources

1. The Software Carpentry git-novice tutorial covers the basics of *Git*:
   - setting up and using *Git*
   - collaborating with others using *Git*
   - managing repository history and conflicts
   - the typical and essential use of *Git* commands
2. There is also a ***free*** book on professional *Git* usage.
3. *GitHub* publishes a series of documents called *Git Guide*